Package ‘ndm’

March 6, 2026

Title Neural Disease Modeling Software

Version 0.0.1

Author Connor T. Jerzak [aut, cre]

Maintainer Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

Description Provides an R-first interface for neural disease modeling with a reticulate-managed JAX backend. The package includes backend bootstrap helpers, built-in epidemiological model specifications, TFRecord-consuming data interfaces, and a vendored runtime bundle for running extracted model build and training code entirely from within the package.

Depends R (>= 4.1.0)

Imports data.table,
rlang,
reticulate,
utils

Suggests testthat (>= 3.0.0)

License GPL-3

Encoding UTF-8

LazyData false

Config/testthat/edition 3

RoxygenNote 7.3.3

Contents

`ndm-package` *Neural Disease Modeling Software*

Description

`ndm` is an R-first package for neural disease modeling with a reticulate-managed JAX backend. The package currently provides backend setup, epidemic model spec import/export, TFRecord readers, and a vendored phase-1 runtime extracted from the original `CovidSuperlearner/Analysis` codebase.

Details

The current implementation preserves both historical model families, `"DecoderOnly"` and `"NeuralODE"`, while defaulting to `"DecoderOnly"`. The supported backbone is the transformer path only. Latent attention and Mamba are intentionally excluded.

The package treats the structured `ndm_model_spec` object as the canonical internal representation and supports `.tex` import/export for compatibility with the original epidemic specification workflow.

Key functions

- `ndm_create_config`
- `ndm_build_backend`
- `ndm_initialize_backend`
- `ndm_model_spec`
- `ndm_model_spec_from_tex`
- `ndm_model_spec_to_tex`
- `ndm_read_tfrecord_dataset`
- `ndm_load_tfrecord_bundle`
- `ndm_prepare_runtime`
- `ndm_prepare_data`
- `ndm_build_model`
- `ndm_train`
- `ndm_predict`
- `ndm_loss`
- `ndm_fit`

Author(s)

Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

ndm_build_model *Build and train models with the vendored runtime*

Description

These wrappers layer a small R API over the extracted Phase 1 model build and training scripts.

Usage

```
ndm_build_model(
  runtime_env,
  analysis_root = .ndm_default_analysis_root(),
  model_type = c("DecoderOnly", "NeuralODE"),
  model_spec = NULL,
  backbone = "transformer",
  runtime_globals = list()
)

## S3 method for class 'ndm_model'
print(x, ...)

ndm_train(x, analysis_root = NULL, run_define = TRUE, run_loop = TRUE)

## S3 method for class 'ndm_trained_model'
print(x, ...)

ndm_fit(
  config = ndm_create_config(),
  model_spec = NULL,
  data_generator = c("sim", "real"),
  runtime_globals = list(),
  data_globals = list(),
  build_globals = list(),
  train_globals = list(),
  run_define = TRUE,
  run_loop = TRUE
)
```

Arguments

runtime_env	Runtime environment containing the sourced legacy helper code and data globals.
analysis_root	Root directory for the legacy analysis runtime. The current implementation resolves runtime files from the vendored package bundle.
model_type	Model family to build. Either ‘"DecoderOnly"‘ or ‘"NeuralODE"‘.
model_spec	Optional ‘ndm_model_spec‘ object used to override the model TeX specification supplied to the vendored builder.
backbone	Backbone family. Phase 1 supports ‘"transformer"‘ only.

`runtime_globals`
: Named list of additional globals assigned during the relevant runtime stage. ‘ndm_build_model()‘ uses them before building the model, while ‘ndm_fit()‘ uses them before sourcing the vendored runtime.

`x`
: Either an ‘ndm_model‘, an ‘ndm_trained_model‘, or a prepared runtime environment, depending on the function being called.

`...`
: Unused; included for S3 method compatibility.

`run_define`
: Logical scalar indicating whether the training definition script should be sourced.

`run_loop`
: Logical scalar indicating whether the training loop script should be sourced.

`config`
: An object of class ‘ndm_config‘, usually created by ‘ndm_create_config()‘.

`data_generator`
: Which extracted data generator to source before calling ‘ndm_build_model()‘ inside ‘ndm_fit()‘.

`data_globals`
: Named list of globals assigned before sourcing the data generator in ‘ndm_fit()‘.

`build_globals`
: Named list of globals assigned before building the model in ‘ndm_fit()‘.

`train_globals`
: Named list of globals assigned immediately before training in ‘ndm_fit()‘.

Value

‘ndm_build_model()‘ returns an object of class ‘ndm_model‘. ‘ndm_train()‘ and ‘ndm_fit()‘ return an object of class ‘ndm_trained_model‘.

‘print.ndm_model()‘ prints a brief summary of an ‘ndm_model‘ object and invisibly returns ‘x‘.

‘print.ndm_trained_model()‘ prints a brief summary of an ‘ndm_trained_model‘ object and invisibly returns ‘x‘.

Examples

```
cfg <- ndm_create_config()
spec <- ndm_model_spec()
## Not run:
runtime_env <- ndm_prepare_runtime(cfg)
ndm_prepare_data(runtime_env, generator = "sim")
model <- ndm_build_model(runtime_env, model_spec = spec)
trained <- ndm_train(model)

## End(Not run)
```

`ndm_check_backend` *Provision and initialize the Python backend*

Description

These helpers manage the reticulate-backed Python environment used by ndm. Use ‘ndm_build_backend()‘ to provision dependencies, ‘ndm_check_backend()‘ to verify availability, and ‘ndm_initialize_backend()‘ to import the active JAX stack into R.

Usage

```
ndm_check_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  modules = c("jax", "numpy", "optax", "equinox", "diffrax")
)

ndm_build_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  python_version = "3.13",
  include_tensorflow = TRUE,
  extra_packages = c("optax", "equinox", "diffrax")
)

ndm_initialize_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  conda_env_required = TRUE,
  float_type = c("32", "64"),
  import_tensorflow = TRUE
)

ndm_backend_modules()
```

Arguments

`conda_env`	Name of the conda environment that should contain the JAX runtime.
`conda`	Conda executable to use. ‘"auto"‘ delegates discovery to reticulate.
`modules`	Character vector of Python module names that must be importable for the backend to be considered healthy.
`python_version`	Python version requested when creating the conda environment.
`include_tensorflow`	Logical scalar indicating whether TensorFlow should be installed alongside JAX.
`extra_packages`	Additional Python packages to install with ‘pip‘.
`conda_env_required`	Passed through to ‘reticulate::use_condaenv()‘ as the ‘required‘ flag.
`float_type`	Floating point precision used when configuring JAX. Either ‘"32"‘ or ‘"64"‘.
`import_tensorflow`	Logical scalar indicating whether TensorFlow should be imported when it is available in the selected environment.

Value

‘ndm_check_backend()‘ returns ‘TRUE‘ invisibly when the requested environment and modules are available. It returns ‘NULL‘ after printing a diagnostic message when the backend is not ready.

‘ndm_build_backend()‘ invisibly returns the conda environment name after attempting to provision the requested packages.

‘ndm_initialize_backend()‘ returns an object of class ‘ndm_backend‘. The returned list contains imported Python modules, the configured float type, and a small compatibility shim used by the vendored runtime.

‘ndm_backend_modules()‘ returns the currently initialized ‘ndm_backend‘ object.

Examples

```
ndm_check_backend()

## Not run:
ndm_build_backend()

## End(Not run)

## Not run:
backend <- ndm_initialize_backend()
backend$default_backend

## End(Not run)

## Not run:
ndm_initialize_backend()
ndm_backend_modules()

## End(Not run)
```

---

ndm_create_config — *Create runtime configuration objects*

Description

Configuration objects collect the runtime options that are threaded through the higher-level orchestration helpers such as ‘ndm_prepare_runtime()‘ and ‘ndm_fit()‘.

Usage

```
ndm_create_config(
  model_type = c("DecoderOnly", "NeuralODE"),
  backbone = "transformer",
  analysis_root = .ndm_default_analysis_root(),
  float_type = c("32", "64"),
  force_to_gpu = TRUE,
  resave_tfrecords = FALSE,
  gpu_mem_frac = NULL,
  ...
)

## S3 method for class 'ndm_config'
print(x, ...)
```

Arguments

model_type	Model family metadata. Use ‘"DecoderOnly"‘ or ‘"NeuralODE"‘.
backbone	Backbone family. Phase 1 supports ‘"transformer"‘ only.
analysis_root	Optional analysis root recorded on the configuration object for compatibility with the historical runtime interface.
float_type	Floating point precision used when initializing the backend. Use ‘"32"‘ or ‘"64"‘.
force_to_gpu	Logical scalar indicating whether the runtime should try to place arrays on GPU when available.
resave_tfrecords	Logical scalar preserved for compatibility with the extracted runtime.
gpu_mem_frac	Optional GPU memory fraction forwarded into the vendored runtime bootstrap code.
...	Unused; included for S3 method compatibility.
x	An ‘ndm_config‘ object.

Value

‘ndm_create_config()‘ returns an object of class ‘ndm_config‘.

‘print.ndm_config()‘ prints a brief summary of an ‘ndm_config‘ object and invisibly returns ‘x‘.

Examples

```
cfg <- ndm_create_config()
print(cfg)
```

---

ndm_create_tfrecord_parser

Create and read TensorFlow TFRecord datasets

Description

These wrappers construct parsers for the ndm TFRecord schema and use them to build TensorFlow datasets or eagerly collect batches into R.

Usage

```
ndm_create_tfrecord_parser(field_names, dtype_map = NULL, tensorflow = NULL)

ndm_read_tfrecord_dataset(
  file,
  batch_size,
  field_names,
  dtype_map = NULL,
  max_examples = NULL,
  shuffle = FALSE,
  buffer_scaler = 10L,
  tensorflow = NULL
```

```
)

ndm_collect_tfrecord_batches(
  file,
  batch_size,
  field_names,
  dtype_map = NULL,
  max_batches = NULL,
  max_examples = NULL,
  shuffle = FALSE,
  buffer_scaler = 10L,
  tensorflow = NULL
)
```

Arguments

`field_names`	Character vector of TFRecord field names that should be parsed from each example.
`dtype_map`	Optional named vector or list mapping ‘field_names‘ to TensorFlow dtype names or TensorFlow dtype objects.
`tensorflow`	Optional TensorFlow module. When omitted, ndm uses the active backend’s TensorFlow module when available and otherwise imports TensorFlow from the active reticulate environment at call time.
`file`	Path to the ‘.tfrecord‘ file to read.
`batch_size`	Batch size used when constructing the TensorFlow dataset.
`max_examples`	Optional maximum number of examples to consume from the file before batching.
`shuffle`	Logical scalar indicating whether the dataset should be shuffled.
`buffer_scaler`	Integer multiplier used to derive the TensorFlow shuffle buffer size from ‘batch_size‘.
`max_batches`	Optional maximum number of batches to collect when using ‘ndm_collect_tfrecord_batches()‘.

Value

‘ndm_create_tfrecord_parser()‘ returns a parser function compatible with ‘tf$data$Dataset$map()‘. ‘ndm_read_tfrecord_dataset()‘ returns a TensorFlow dataset object. ‘ndm_collect_tfrecord_batches()‘ returns a list of parsed batches.

Examples

```
fields <- ndm_tfrecord_fields_real()
parser <- ndm_create_tfrecord_parser(fields, ndm_tfrecord_dtype_map("real"))
is.function(parser)
```

`ndm_model_spec_presets` *Inspect and create epidemic model specifications*

Description

The package ships a small registry of built-in compartmental model specifications and also supports importing custom '.tex' model definitions from disk. The structured 'ndm_model_spec' object is the canonical representation used by the runtime wrappers.

Usage

```
ndm_model_spec_presets()

ndm_model_spec(
  preset = NULL,
  compartments = c("seir", "seirs"),
  dynamic_beta = NULL,
  dynamic_global = FALSE,
  multi_outcome = FALSE,
  model_type = c("DecoderOnly", "NeuralODE"),
  time_varying_terms = NULL,
  endogenous_terms = NULL,
  tex_path = NULL
)

ndm_model_spec_from_tex(path, model_type = c("DecoderOnly", "NeuralODE"))

ndm_model_spec_to_tex(spec, path = NULL)

## S3 method for class 'ndm_model_spec'
print(x, ...)
```

Arguments

`preset`	Optional preset name from 'ndm_model_spec_presets()'. When omitted, the remaining arguments are used to infer a built-in preset.
`compartments`	Compartment family used when inferring a built-in preset.
`dynamic_beta`	Logical scalar controlling whether the local transmission rate is time-varying for built-in presets.
`dynamic_global`	Logical scalar controlling whether the global rates are time-varying for built-in presets.
`multi_outcome`	Logical scalar indicating whether the observation model should include multiple outcomes.
`model_type`	Model family metadata attached to the returned specification. Either '"DecoderOnly"' or '"NeuralODE"'.
`time_varying_terms`	Optional character vector overriding the default time-varying term metadata stored on the returned spec.

`endogenous_terms`
: Optional character vector overriding the default endogenous term metadata stored on the returned spec.

`tex_path`
: Optional path to a custom ‘.tex‘ file. When supplied, ‘ndm_model_spec()‘ delegates to ‘ndm_model_spec_from_tex()‘.

`path`
: Path to a ‘.tex‘ model specification file on disk.

`spec`
: An ‘ndm_model_spec‘ object.

`x`
: An ‘ndm_model_spec‘ object.

`...`
: Unused; included for S3 method compatibility.

Value

‘ndm_model_spec_presets()‘ returns a data frame describing the built-in presets bundled with the package.

‘ndm_model_spec()‘ returns an object of class ‘ndm_model_spec‘. Built-in presets include metadata such as the packaged source path and the normalized TeX contents.

‘ndm_model_spec_from_tex()‘ returns an ‘ndm_model_spec‘ object whose ‘source_path‘ points at the supplied file. When the basename matches a built-in preset, the preset metadata is reused.

‘ndm_model_spec_to_tex()‘ returns the TeX text as a single character string when ‘path‘ is ‘NULL‘. When ‘path‘ is supplied, it writes the specification to disk and invisibly returns the normalized output path.

‘print.ndm_model_spec()‘ prints a brief summary of an ‘ndm_model_spec‘ object and invisibly returns ‘x‘.

Examples

```
ndm_model_spec_presets()

spec <- ndm_model_spec()
print(spec)

spec <- ndm_model_spec()
tex_path <- tempfile(fileext = ".tex")
ndm_model_spec_to_tex(spec, tex_path)
ndm_model_spec_from_tex(tex_path)

spec <- ndm_model_spec()
tex <- ndm_model_spec_to_tex(spec)
nchar(tex) > 0
```

`ndm_new_runtime_env` *Create and populate runtime environments*

Description

These helpers manage isolated environments used to source the vendored model runtime and seed it with R values.

Usage

```
ndm_new_runtime_env(parent = baseenv())

ndm_set_runtime_globals(env, values, overwrite = TRUE)
```

Arguments

parent	Parent environment for a new runtime environment.
env	Runtime environment that should receive new bindings.
values	Named list of values to assign into ‘env‘.
overwrite	Logical scalar indicating whether existing bindings in ‘env‘ should be overwritten.

Value

‘ndm_new_runtime_env()‘ returns an environment of class ‘ndm_runtime_env‘. ‘ndm_set_runtime_globals()‘ invisibly returns ‘env‘.

Examples

```
env <- ndm_new_runtime_env()
ndm_set_runtime_globals(env, list(example_value = 1L))
env$example_value
```

---

ndm_predict *Generate predictions or evaluate the training loss*

Description

These helpers call the vendored prediction and loss functions stored in a prepared runtime environment.

Usage

```
ndm_predict(x, batch = NULL, inference = TRUE, seed = 1L, update_state = TRUE)

ndm_loss(
  x,
  batch = NULL,
  y = NULL,
  y_mask = NULL,
  iteration = 1L,
  seed = 1L,
  update_state = TRUE
)
```

Arguments

x	Either an ‘ndm_model‘, an ‘ndm_trained_model‘, or a prepared runtime environment that already contains the required vendored objects.
batch	Optional batch object. Supply either a named TFRecord-style list or the four-element packaged list returned by ‘ndm_batch_to_model_inputs()‘. When ‘NULL‘, the runtime must already contain ‘batch_l_cal‘.
inference	Logical scalar indicating whether ‘ndm_predict()‘ should use the inference prediction path or the training prediction path.
seed	Integer seed used to derive the per-example JAX RNG keys.
update_state	Logical scalar indicating whether the runtime ‘state‘ object should be updated with the returned state.
y	Optional observed outcome tensor passed to ‘ndm_loss()‘. When omitted, ‘YTrue_out‘ is inferred from ‘batch‘.
y_mask	Optional outcome mask tensor passed to ‘ndm_loss()‘. When omitted, ‘YTrue_out_mask‘ is inferred from ‘batch‘.
iteration	Training iteration number forwarded to the vendored loss function.

Value

‘ndm_predict()‘ returns the prediction object produced by the vendored runtime. ‘ndm_loss()‘ returns the loss object produced by the vendored runtime.

Examples

```
## Not run:
preds <- ndm_predict(model, batch = batch_l_cal)
loss <- ndm_loss(model, batch = batch_l_cal)

## End(Not run)
```

---

ndm_prepare_runtime *Prepare the vendored runtime for execution*

Description

These wrappers load the extracted runtime scripts into an isolated environment and then source the selected data generator.

Usage

```
ndm_prepare_runtime(
  config = ndm_create_config(),
  runtime_env = ndm_new_runtime_env(),
  runtime_globals = list()
)

ndm_prepare_data(
  runtime_env,
```

```
  analysis_root = .ndm_default_analysis_root(),
  generator = c("sim", "real"),
  runtime_globals = list()
)
```

Arguments

config	An object of class ‘ndm_config‘, usually created by ‘ndm_create_config()‘.
runtime_env	Runtime environment that should receive the sourced objects.
runtime_globals	Named list of additional bindings to assign before the runtime code is sourced.
analysis_root	Root directory for the legacy analysis runtime. The current implementation resolves runtime files from the vendored package bundle.
generator	Which extracted data generator to source into ‘runtime_env‘.

Value

‘ndm_prepare_runtime()‘ invisibly returns ‘runtime_env‘ after loading the vendored helper and backend code.

‘ndm_prepare_data()‘ invisibly returns ‘runtime_env‘ after sourcing the requested data generator.

Examples

```
env <- ndm_new_runtime_env()
cfg <- ndm_create_config()
## Not run:
ndm_prepare_runtime(cfg, env)

## End(Not run)

env <- ndm_new_runtime_env()
ndm_set_runtime_globals(env, list(example_value = 1L))
```

---

ndm_print — *Print a timestamped diagnostic message*

Description

‘ndm_print()‘ is a small logging helper used throughout the package-facing orchestration code.

Usage

```
ndm_print(text, quiet = FALSE)
```

Arguments

text	Text to print.
quiet	Logical scalar indicating whether output should be suppressed.

Value

The input ‘text‘, invisibly.

Examples

```
ndm_print("starting")
```

---

ndm_runtime_paths *Inspect the vendored runtime bundle*

Description

ndm vendors the active Phase 1 runtime files under ‘inst/extracted‘ so the package can load the historical analysis code without depending on an external checkout.

Usage

```
ndm_runtime_paths(analysis_root = .ndm_default_analysis_root())
```

Arguments

analysis_root	Retained for API compatibility. The current implementation always resolves paths from the vendored runtime bundle.

Value

‘ndm_runtime_paths()‘ returns a named list of normalized file paths for the extracted runtime bundle.

Examples

```
paths <- ndm_runtime_paths()
names(paths)
```

---

ndm_run_analysis2_real *Run the packaged Analysis2 orchestration scripts*

Description

These wrappers expose the migrated Analysis2 entry points from within the R package. They are primarily intended for script-style execution.

Usage

```
ndm_run_analysis2_real(args = commandArgs(TRUE))

ndm_run_analysis2_sim(args = commandArgs(TRUE))
```

Arguments

`args` Character vector of command line style arguments, usually of the form ‘"–key=value"‘.

Value

The invisible result returned by the underlying Analysis2 driver.

Examples

```
## Not run:
ndm_run_analysis2_real(c("--dry_run=TRUE"))
ndm_run_analysis2_sim(c("--dry_run=TRUE"))

## End(Not run)
```

---

ndm_source_runtime_helper_fxns

Source vendored runtime components

Description

These helpers source the extracted legacy runtime files into a dedicated environment. They are lower-level building blocks used by ‘ndm_prepare_runtime()‘ and ‘ndm_fit()‘.

Usage

```
ndm_source_runtime_helper_fxns(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env()
)

ndm_source_runtime_backend(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env(),
  float_type = "32",
  force_to_gpu = TRUE,
  gpu_mem_frac = NULL,
  resave_tfrecords = FALSE
)

ndm_load_runtime(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env(),
  float_type = "32",
  force_to_gpu = TRUE,
  gpu_mem_frac = NULL,
  resave_tfrecords = FALSE
)
```

```
ndm_source_runtime_data(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env(),
  generator = c("sim", "real")
)

ndm_source_runtime_calibration(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env()
)

ndm_source_runtime_results_get(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env()
)

ndm_source_runtime_results_analyze(
  analysis_root = .ndm_default_analysis_root(),
  env = ndm_new_runtime_env()
)
```

Arguments

analysis_root	Retained for API compatibility. The current implementation always resolves paths from the vendored runtime bundle.
env	Runtime environment that should receive the sourced objects.
float_type	Floating point precision passed into the vendored backend bootstrap code. Use ‘"32"‘ or ‘"64"‘.
force_to_gpu	Logical scalar indicating whether the runtime should try to place arrays on a GPU device when available.
gpu_mem_frac	Optional GPU memory fraction forwarded to the vendored backend bootstrap code.
resave_tfrecords	Logical scalar preserved for compatibility with the legacy runtime setup.
generator	Which extracted data generator script to source.

Value

Each function on this page invisibly returns ‘env‘ after sourcing the requested runtime component.

Examples

```
## Not run:
env <- ndm_new_runtime_env()
ndm_load_runtime(env = env)

## End(Not run)
```

ndm_tfrecord_fields_real — *Inspect TFRecord field contracts*

Description

These helpers describe the field names and default TensorFlow dtypes expected by the packaged TFRecord readers.

Usage

```
ndm_tfrecord_fields_real()

ndm_tfrecord_fields_sim()

ndm_tfrecord_dtype_map(kind = c("real", "sim"))
```

Arguments

`kind`	TFRecord schema to inspect. Use ‘"real"‘ for observed-data records and ‘"sim"‘ for simulation records.

Value

‘ndm_tfrecord_fields_real()‘ and ‘ndm_tfrecord_fields_sim()‘ return character vectors of field names. ‘ndm_tfrecord_dtype_map()‘ returns a named character vector mapping each field to its default TensorFlow dtype.

Examples

```
ndm_tfrecord_fields_real()
ndm_tfrecord_dtype_map("sim")
```

ndm_tf_batch_to_r — *Convert and bundle TFRecord batches*

Description

These helpers convert TensorFlow batches to native R or JAX objects, package them into the shape expected by the vendored runtime, and attach dataset handles to a runtime environment.

Usage

```
ndm_tf_batch_to_r(batch)

ndm_tf_batch_to_jax(batch, backend = NULL)

ndm_batch_to_model_inputs(batch)
```

```
ndm_load_tfrecord_bundle(
  train_file,
  inference_file = NULL,
  batch_size,
  kind = c("real", "sim"),
  dtype_map = NULL,
  tensorflow = NULL,
  max_train_examples = NULL,
  max_inference_examples = NULL,
  shuffle_train = TRUE,
  shuffle_inference = FALSE
)

ndm_attach_tfrecord_bundle(
  env,
  bundle,
  backend = NULL,
  calibration_source = c("inference", "train")
)
```

Arguments

batch	A parsed TFRecord batch or a nested list containing TensorFlow tensors.
backend	Optional 'ndm_backend' object. When omitted, 'ndm_tf_batch_to_jax()' and 'ndm_attach_tfrecord_bundle()' use the active backend from 'ndm_backend_modules()' when available.
train_file	Path to the training TFRecord file.
inference_file	Optional path to an inference TFRecord file.
batch_size	Batch size used when constructing the training and inference datasets.
kind	TFRecord schema to use when reading 'train_file' and 'inference_file'.
dtype_map	Optional named dtype mapping passed through to the TFRecord readers.
tensorflow	Optional TensorFlow module. When omitted, ndm uses the active backend's TensorFlow module when available and otherwise imports TensorFlow from the active reticulate environment at call time.
max_train_examples	Optional cap on the number of training examples to read.
max_inference_examples	Optional cap on the number of inference examples to read.
shuffle_train	Logical scalar indicating whether the training dataset should be shuffled.
shuffle_inference	Logical scalar indicating whether the inference dataset should be shuffled.
env	Runtime environment that should receive dataset handles and the preview calibration batch.
bundle	An object of class 'ndm_tfrecord_bundle' returned by 'ndm_load_tfrecord_bundle()'.
calibration_source	Which dataset should supply the preview batch used to seed runtime globals such as 'batch_l_cal'.

Value

‘ndm_tf_batch_to_r()‘ recursively converts a TensorFlow batch to R arrays and lists. ‘ndm_tf_batch_to_jax()‘ recursively converts tensors into JAX arrays. ‘ndm_batch_to_model_inputs()‘ returns the packaged four-element list expected by the vendored model code. ‘ndm_load_tfrecord_bundle()‘ returns an ‘ndm_tfrecord_bundle‘ object. ‘ndm_attach_tfrecord_bundle()‘ invisibly returns ‘env‘.

Examples

```
batch <- list(
  XPred = array(0, c(2, 3, 4)),
  XPred_mask = array(TRUE, c(2, 3, 4)),
  Context = array(0, c(2, 1, 4)),
  Context_mask = array(TRUE, c(2, 1, 4)),
  location_id_numeric = matrix(1L, nrow = 2),
  time_id_numeric = matrix(1L, nrow = 2)
)
packaged <- ndm_batch_to_model_inputs(batch)
length(packaged)
```

Index